

Sequence Learning - 2

Dr. Souvik Chakraborty

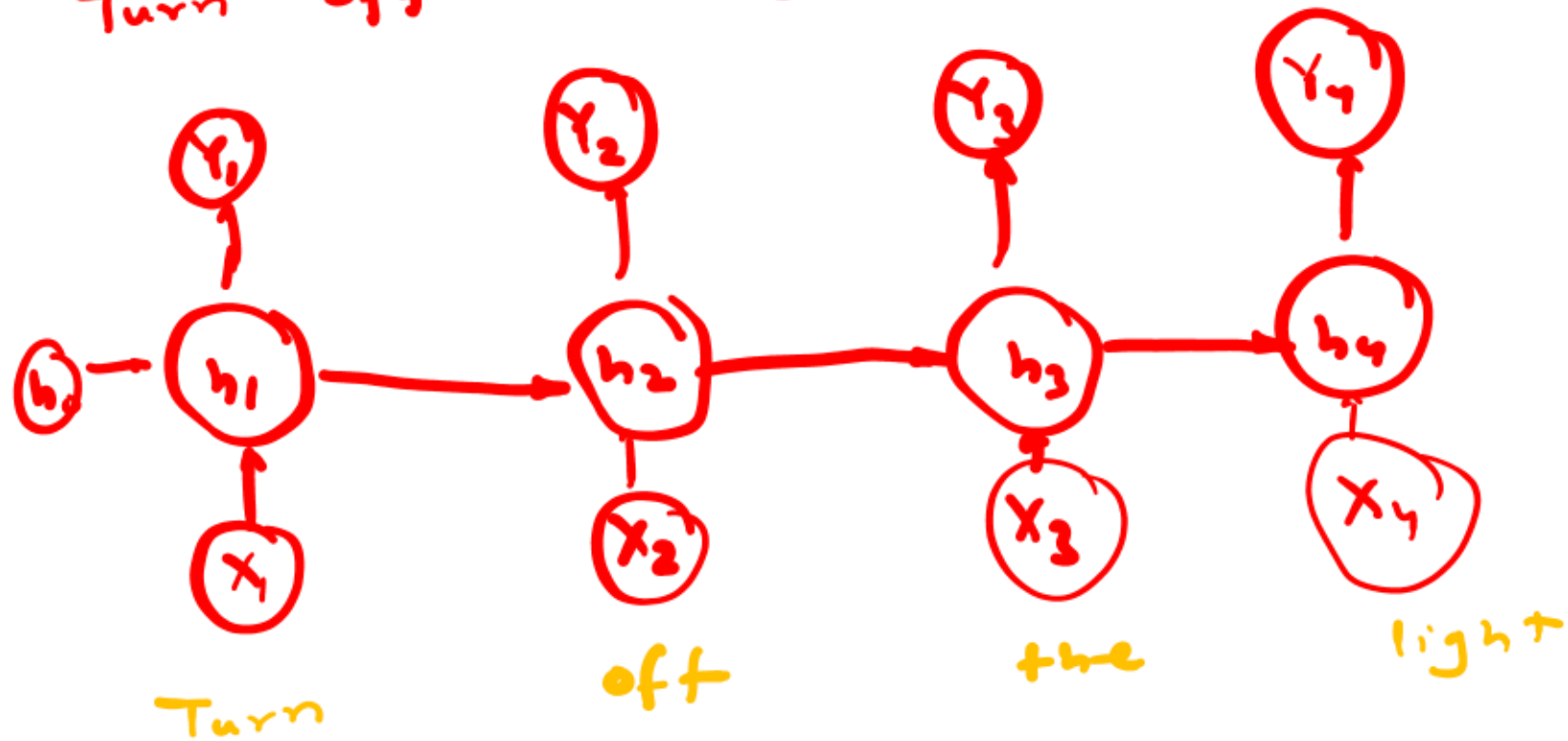
Department of Applied Mechanics
Yardi School of Artificial Intelligence (joint appointment)
Indian Institute of Technology Delhi
Hauz Khas – 110016, Delhi, India.

E-mail: souvik@am.iitd.ac.in

Website: <https://www.csccm.in/>

* We were talking about Seq2Seq Learning
↳ Machine translation.

Turn off the light →



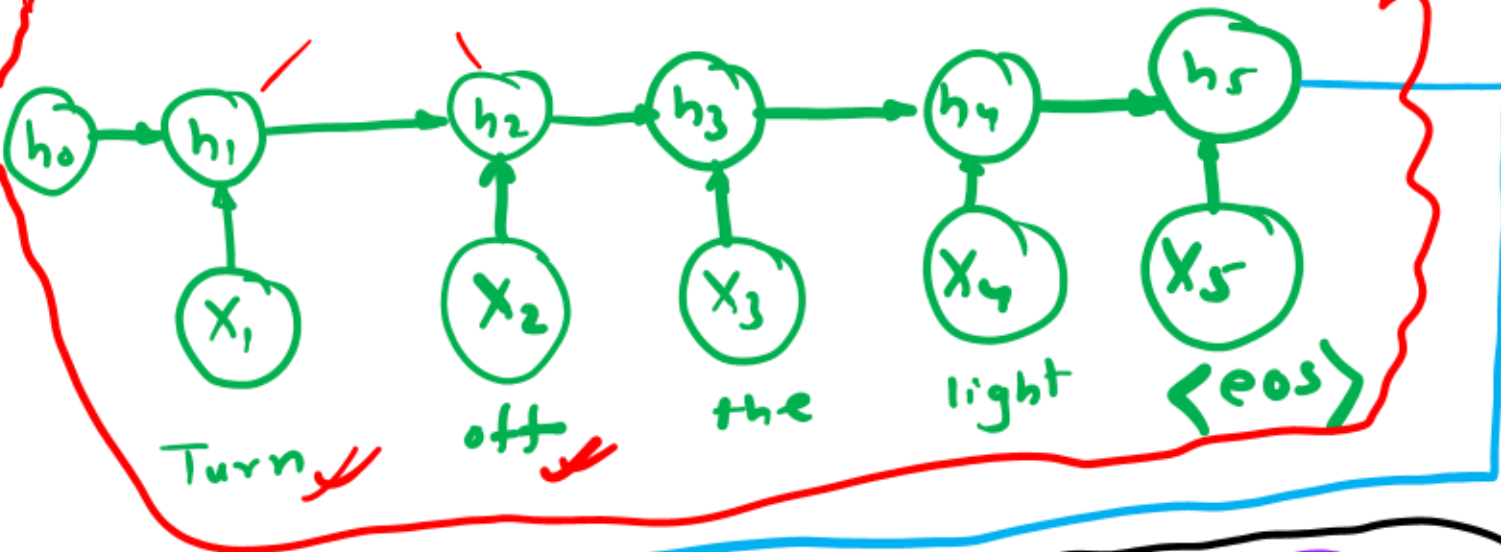
* Clearly, RNN/ LSTM in its raw form won't work.

→ The model should process the input before attempting to predict / provide the output.

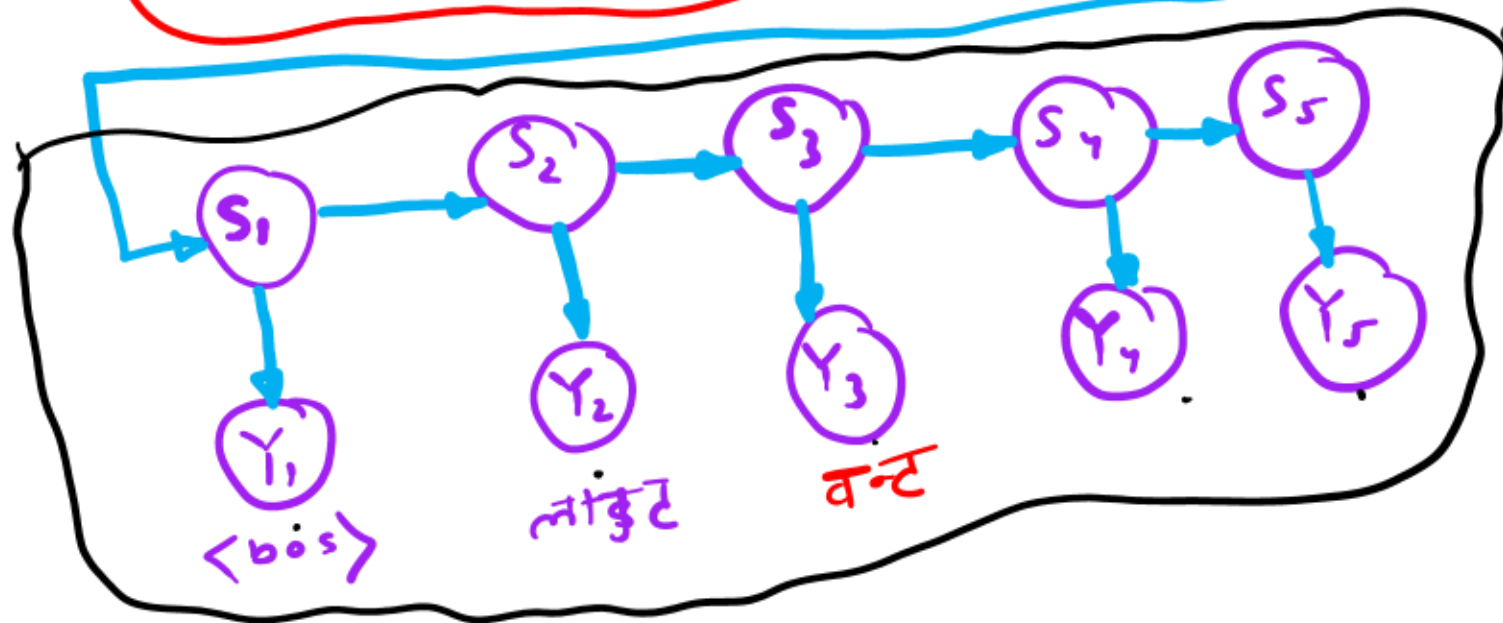
↓
Motivated development of
Encoder - Decoder architecture.

Encoder - Decoder Architecture

Encoder

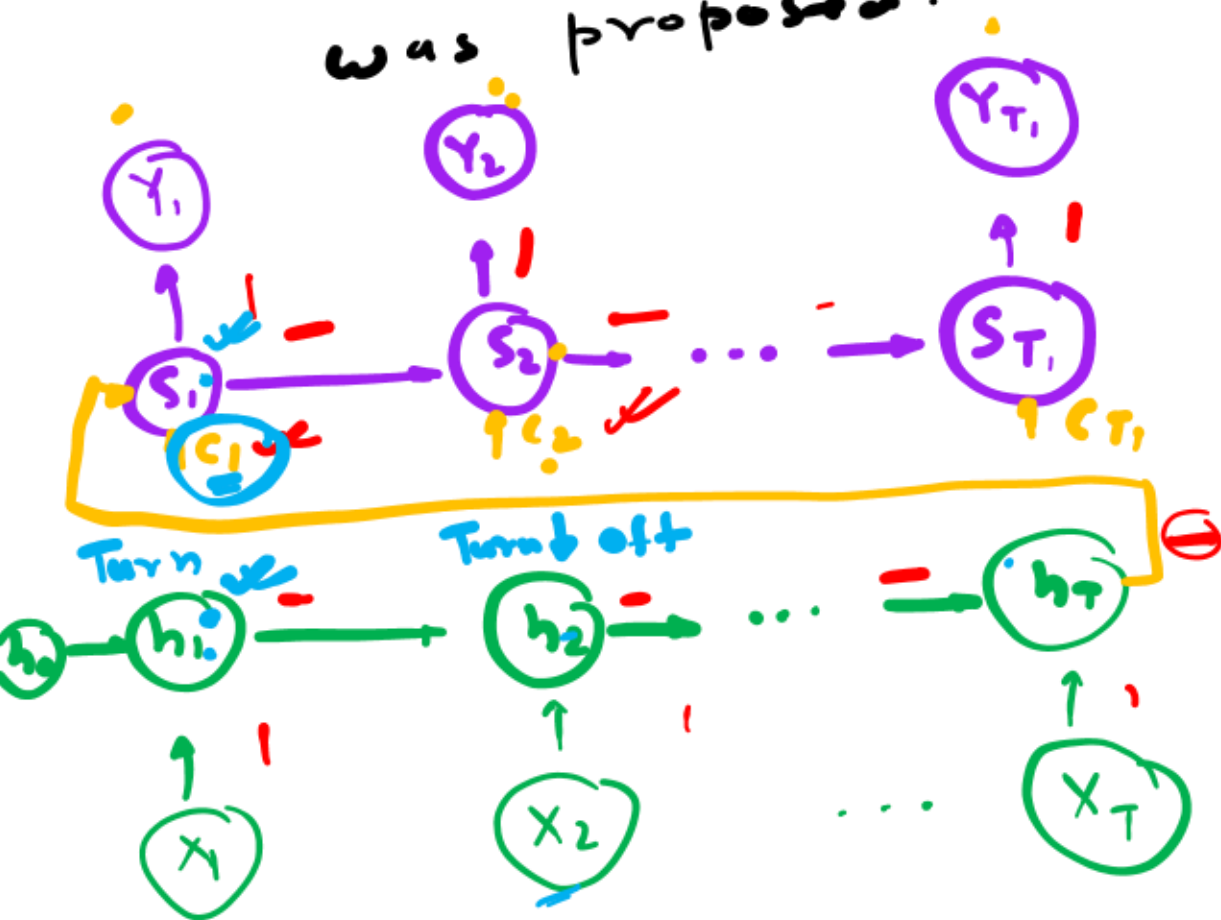


Decoder



Encoder - Decoder Architecture Struggles with long sentences.

↳ To remedy this, attention mechanism was proposed.



$c_i \rightarrow$ measures similarity between

s_i and

h

$$C_1 = \begin{bmatrix} \end{bmatrix}$$

$$x_0 = \begin{bmatrix} \text{Turn off the light} \end{bmatrix}$$

$$h = \begin{bmatrix} h_1 \end{bmatrix}$$

$$Y = \begin{bmatrix} \text{नाइट} \end{bmatrix}$$

$$s = \begin{bmatrix} s_1 \end{bmatrix}$$

$$C_1 = \begin{bmatrix} 0.1 \\ 0.6 \end{bmatrix}$$

$$\begin{bmatrix} s_2 \\ s_3 \end{bmatrix}$$

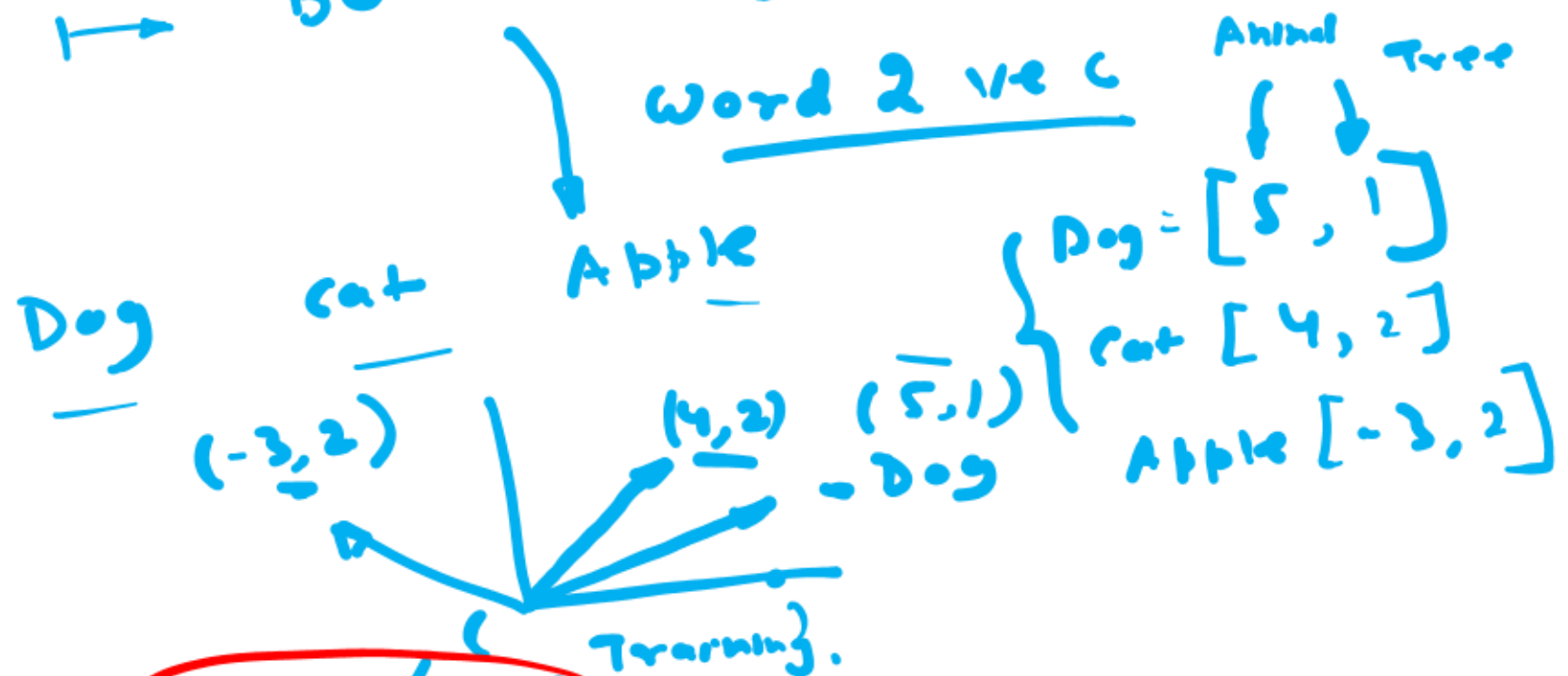
$$\begin{bmatrix} 0.05 & 0.05 & 0.7 \\ 0.01 & 0.09 \end{bmatrix}$$

$$C = f(h, s) = \sigma(w^T x)$$

Word Embeddings :-

→ one hot encoding

→ Bow → bag of words.



$$y = \sigma(w^T x + b)$$

fixed

* Transformer :-

* A key component of transformer is
Self-attention.

Need for Self-attention :-

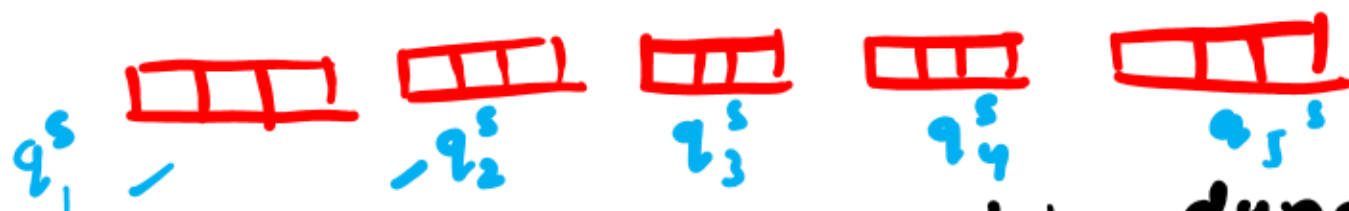
Word embedding learns vector
representation of words in

"average" sense.

↳ Static

* Self-attention converts static embedding to dynamic.

* I love to eat Apples. } word2vec.
Static



* One way to develop dynamic embedding is to make the function of all the words in the sentence.

$$q_1^d = \underline{c_{11}} q_1^s + c_{12} q_2^s + c_{13} q_3^s + c_{14} q_4^s + c_{15} q_5^s$$

$c_{ij} \leftarrow$ attention weight

↓
measure similarity.

↓
one way of
measuring similarity
is dot product.

↓
Not between $[0, 1]$

↓
Use softmax

$$c_{ij} = \text{Softmax}(q_i^s q_j^{sT})$$

$$\underline{q_1^d} = \text{softmax} \left(\begin{array}{c} \text{[vector]} \\ q_1^s \end{array} \begin{array}{c} \text{[vector]} \\ q_1^{s^T} \end{array} \right) \underline{q_1^s} +$$

$$\text{softmax} \left(\begin{array}{c} \text{[vector]} \\ q_1^s \end{array} \begin{array}{c} \text{[vector]} \\ q_2^{s^T} \end{array} \right) \underline{q_2^s} + \dots$$

$$\text{softmax} = \frac{\exp(x_i)}{\sum_{j=1}^N \exp(x_j)}$$

Query , key , value

$$\underline{q_v^i} = f(\underline{q_i^s}),$$

[] [q_i^s]

$$k_v^i = g(q_i^s),$$

$$v_v^i = h(q_i^s)$$

Static embedding $\equiv$ Diary



Self-attention on q, k, v

$$\left[\text{softmax} (q k^T) v \right]$$

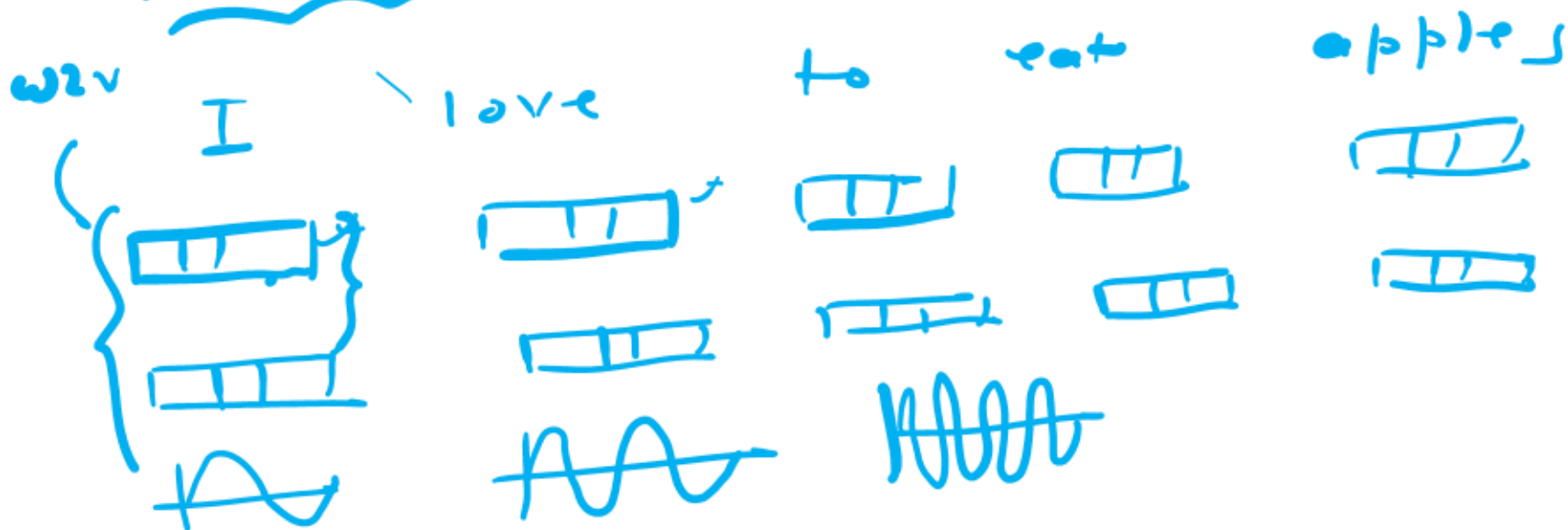
Self-attention operation.

w_q, w_k, w_v

weights corresponding to query,

key, value.

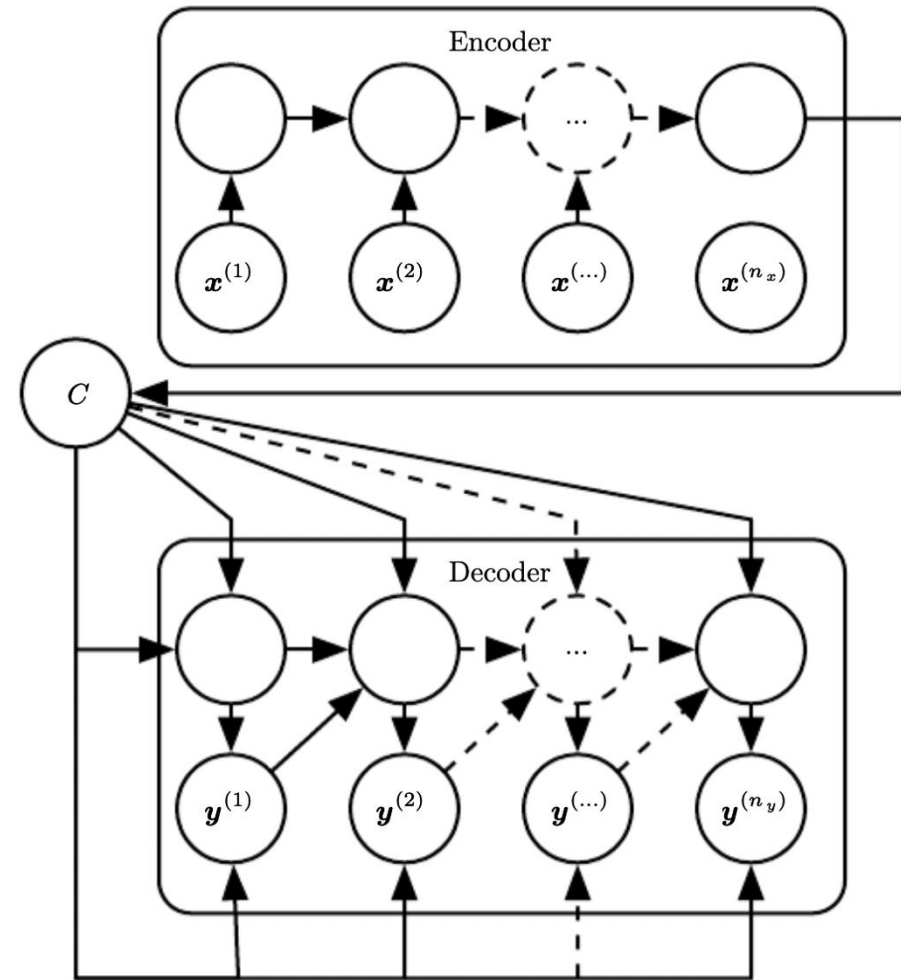
positional Embedding :-



Encoder-Decoder Sequence-to-Sequence Architectures

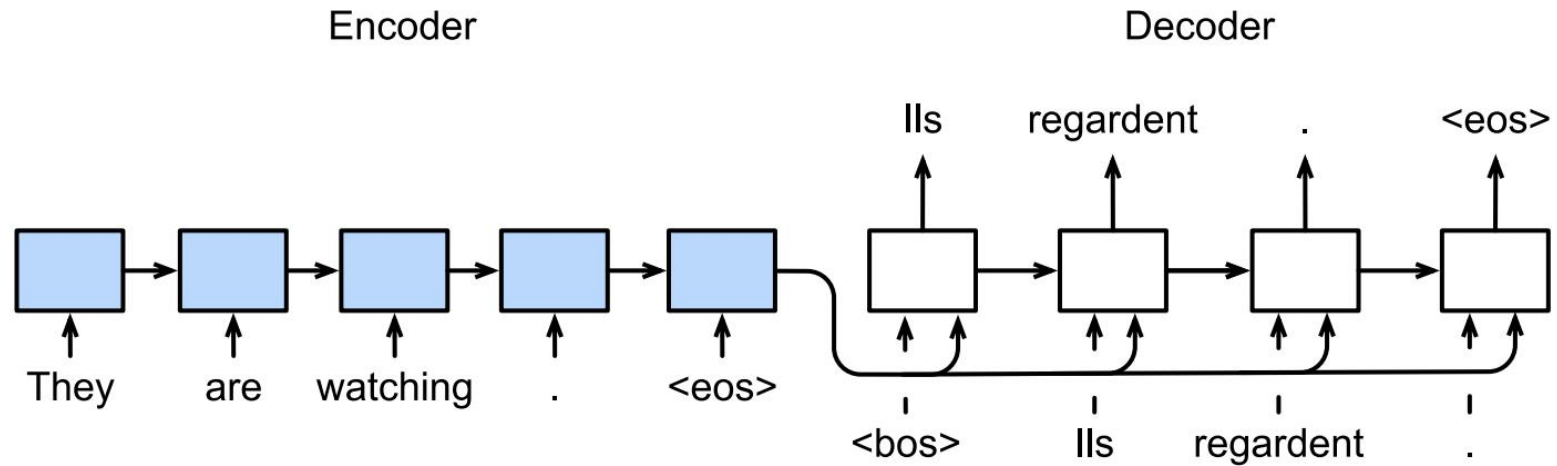
- Now we discuss how an RNN can be trained to map an input sequence to an output sequence which is not necessarily of the same length. This comes up in many applications, such as speech recognition, machine translation and question answering, where the input and output sequences in the training set are generally not of the same length (although their lengths might be related).
- We often call the input to the RNN the “context.” We want to produce a representation of this context, C . The context C might be a vector or sequence of vectors that summarize the input sequence $\mathbf{X} = (\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(n_x)})$.
- The idea is very simple:
 - (1) An encoder or reader or input RNN processes the input sequence. The encoder emits the context C , usually as a simple function of its final hidden state.
 - (2) A decoder or writer or output RNN is conditioned on that fixed-length to generate the output sequence $\mathbf{Y} = (\mathbf{y}^{(1)}, \dots, \mathbf{y}^{(n_y)})$.

Encoder-Decoder Sequence-to-Sequence Architectures



- Example of an encoder-decoder or sequence-to-sequence RNN architecture, for learning to generate an output sequence $(\mathbf{y}^{(1)}, \dots, \mathbf{y}^{(n_y)})$ given an input sequence $(\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \dots, \mathbf{x}^{(n_x)})$.
- It is composed of an encoder RNN that reads the input sequence as well as a decoder RNN that generates the output sequence (or computes the probability of a given output sequence).
- The final hidden state of the encoder RNN is used to compute a generally fixed-size context variable C , which represents a semantic summary of the input sequence and is given as input to the decoder RNN.

Encoder-Decoder Sequence-to-Sequence Architectures

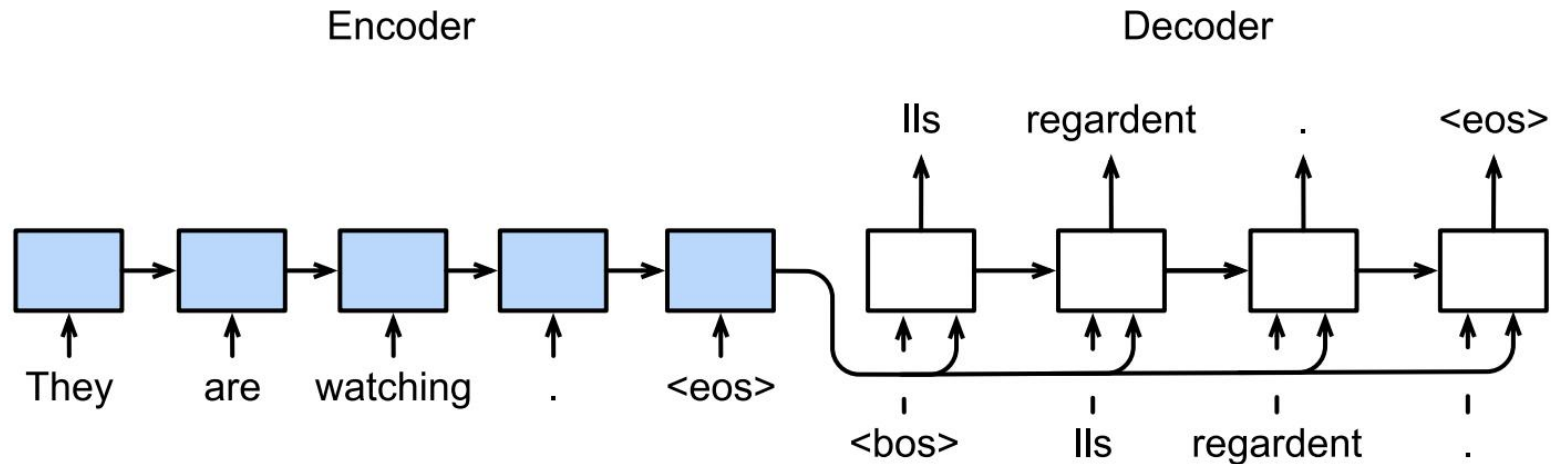


Sequence to sequence learning with an RNN encoder and an RNN decoder.

The special “<eos>” token marks the end of the sequence. The model can stop making predictions once this token is generated. At the initial time step of the RNN decoder, there are two special design decisions. First, the special beginning-of-sequence “<bos>” token is an input.

- A. Zhang, Z. Lipton, M. Li, and A. Smola. [Dive into deep learning](#). 2019.

Encoder- decoder structure



The ancient library at the center of the old city contained thousands of handwritten manuscripts describing forgotten kingdoms, astronomical discoveries, medicinal herbs, philosophical debates, and mathematical theories that had been collected over centuries by travelers, scholars, merchants, monks, and historians from distant civilizations, but when the encoder-decoder model attempted to summarize the entire passage into a fixed-length context vector before decoding, it gradually forgot important information from the beginning of the sentence, demonstrating the bottleneck problem of traditional encoder-decoder architectures

Attention

- In all of the neural networks we have considered so far, the hidden activations are a linear combination of the input activations, followed by a nonlinearity:

$$z = \varphi(\mathbf{w}^\top \mathbf{h})$$

- The weights $\mathbf{w}$ are fixed parameters that are learned on a training set and applied to dynamically computed hidden states $\mathbf{h}$.
- Here, we consider models in which the **weight vectors are also computed dynamically, in an input-dependent way**. That is, we consider models which use the following kind of unit:

$$z = \varphi(f(\mathbf{h}; \boldsymbol{\theta})^\top \mathbf{h})$$

where $f(\cdot; \boldsymbol{\theta})$ is **a learnable function that predicts what weights to use at run-time**. This is a form of multiplicative unit. Models with units of the form are said to be using **attention**, since it allows the model to adaptively “*pay attention to*” different parts of its inputs by adjusting the weights.

Seq2seq with attention

- This uses an RNN decoder of the form

$$\mathbf{h}_t^d = f_d(\mathbf{h}_{t-1}^d, \mathbf{c}),$$

where $\mathbf{c}$ is a fixed-length context vector, representing the encoding of the input.

- Usually we set $\mathbf{c} = \mathbf{h}_T^e$, which is the final state of the encoder RNN (or we use a bidirectional RNN with average pooling).
- However, for tasks such as machine translation, this can result in poor performance, since the output does not have access to the input words themselves.
- We can avoid this bottleneck by allowing the output words to directly “look at” the input words.
- But which inputs should it look at?
- After all, word order is not always preserved across languages (e.g., German often puts verbs at the end of a sentence), so we need to infer the alignment between source and target.

Seq2seq with attention

- We can solve this problem (in a differentiable way) by using (soft) attention
- In particular, we let the decoder function be $\mathbf{h}_t^d = f_d(\mathbf{h}_{t-1}^d, \mathbf{c}_t)$, where the context vector is computed using a weighted sum of input encoding vectors,

$$\mathbf{c}_t = \sum_s A_{ts} \mathbf{h}_s^e$$

where A_{ts} are the attention weights given by

$$A_{ts} = \frac{\exp(\text{score}(\mathbf{h}_{t-1}^d, \mathbf{h}_s^e))}{\sum_{s'=1}^{S'} \exp(\text{score}(\mathbf{h}_{t-1}^d, \mathbf{h}_{s'}^e))}$$

- There are several ways to define the score function which measures similarities of the hidden representation. We list two examples below:

$$\text{score}(\mathbf{a}, \mathbf{b}) = \begin{cases} \mathbf{a}^\top \mathbf{W} \mathbf{b} & \text{Luong's multiplicative style} \\ \mathbf{v}^\top \tanh(\mathbf{W}_1 \mathbf{a} + \mathbf{W}_2 \mathbf{b}) & \text{Bahdanau's additive style} \end{cases}$$

Seq2seq with attention

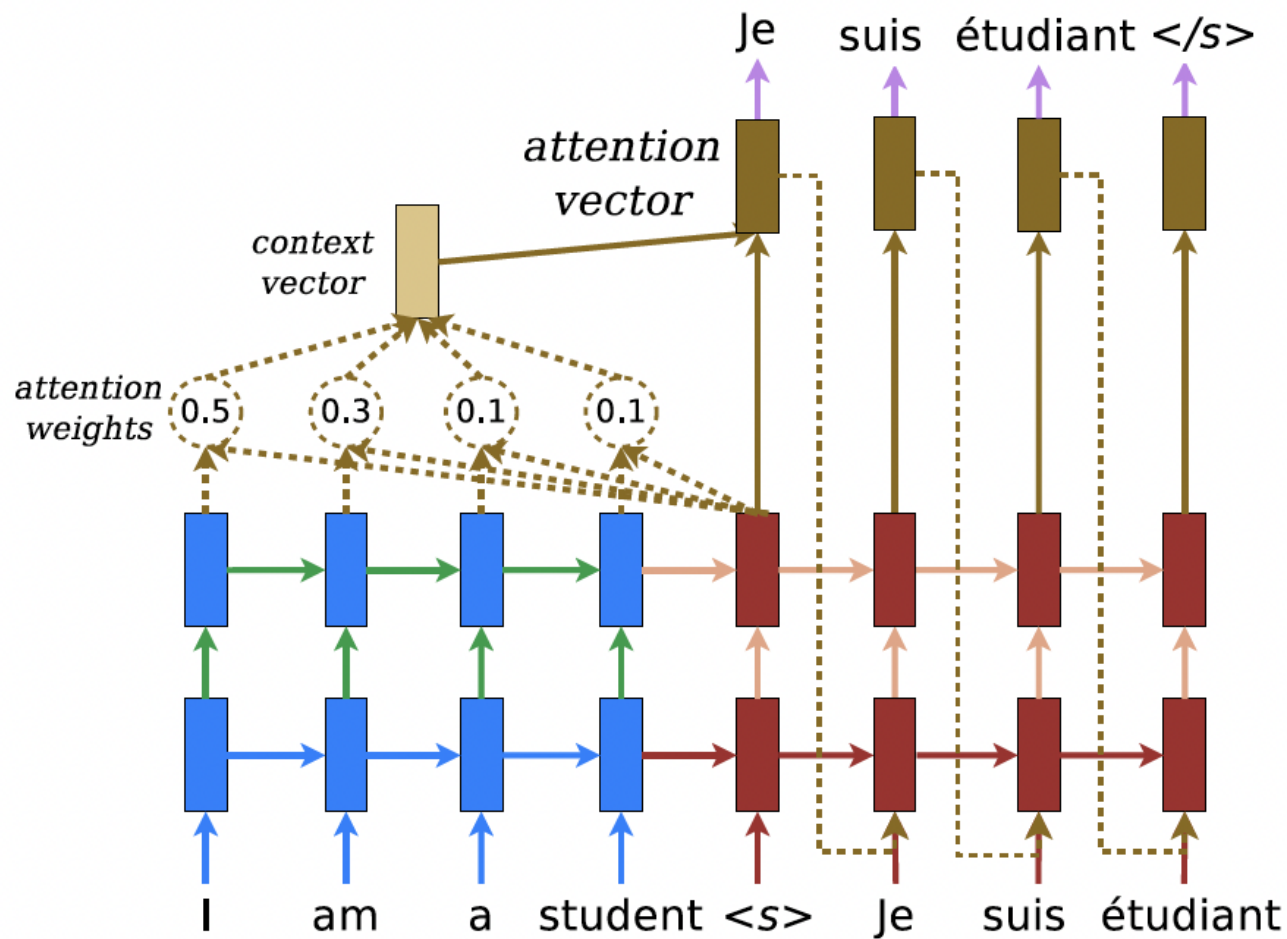
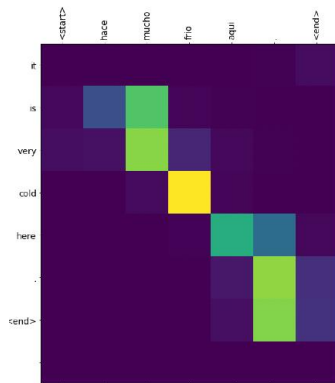


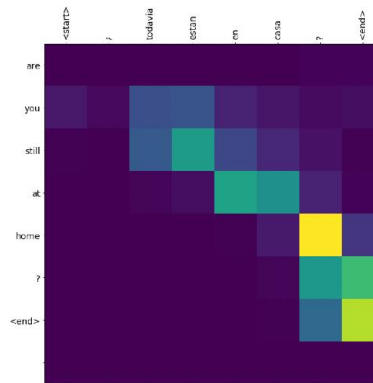
Illustration of seq2seq with attention for English to French translation.

Seq2seq with attention

- We can train this model in the usual way on sentence pairs, and then use it to perform translation.
- We can also visualize the attention weights computed at each step of decoding, to get an idea of which parts of the input the model thinks are most relevant for generating the corresponding output.



(a)



(b)

Illustration of the attention heatmaps generated while translating two sentences from Spanish to English.

(a) Input is “hace mucho frio aqui.”, output is “it is very cold here.”.

(b) Input is “todavia estan en casa?”, output is “are you still at home?”.

Note that when generating the output token “home”, the model should attend to the input token “casa”, but in fact it seems to attend to the input token “?”.

Attention as a soft dictionary lookup

- Earlier, we computed the context vector at output location t , $\mathbf{c}_t$, using a weighted sum of input features from the encoder:

$$\sum_s A_{ts} \mathbf{h}_s^e$$

- However, we can think of attention more generally as comparing a set of target vectors, or queries $\mathbf{q}_i \in \mathbb{R}^D$, with a set of candidate vectors, or keys, $\mathbf{k}_j \in \mathbb{R}^D$. We then compute the attention matrix using:

$$\mathbf{A}_{i,:} = \mathcal{S}([\text{score}(\mathbf{q}_i, \mathbf{k}_j) : j = 1:N])$$

- Given these weights, we compute a weighted combination of the values $\mathbf{v}_j$ associated with each key. In particular, for the i 'th query we get the vector

$$\mathbf{r}_i = \sum_j A_{ij} \mathbf{v}_j$$

Attention as a soft dictionary lookup

- This is a weighted set of values whose keys most resemble the query $\mathbf{q}_i$. We can think of this as a “soft dictionary lookup”. If we store all the values in the rows of a $N \times D$ matrix $\mathbf{V}$, we can write this in matrix form as follows:

$$\mathbf{R} = \text{attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) \triangleq \mathbf{A}\mathbf{V}$$

where $\mathbf{R}$ is the $N \times D$ matrix of retrieved values.

- For the scoring function, it is common to use a simplified version of the Luong score with $\mathbf{W} = \mathbf{1}$.
- Specifically, we use score $(\mathbf{q}, \mathbf{k}) = \langle \mathbf{q}, \mathbf{k} \rangle / \sqrt{D}$, where we divide by $\sqrt{D}$ to ensure the scores have mean 0 and variance 1 (assuming the inputs also have mean 0 and variance 1).
- If $\mathbf{Q} \in \mathbb{R}^{N \times D}$ contains N queries, and $\mathbf{K} \in \mathbb{R}^{N \times D}$ contains N keys, then we can compute all N^2 scores in batch form using
$$\mathbf{S} = \text{score}(\mathbf{Q}, \mathbf{K}) = \mathbf{Q}\mathbf{K}^\top / \sqrt{D}$$
- This is called (scaled) **dot product attention**. We can also use the Bahdanau score (also called MLP attention).

Attention as a soft dictionary lookup

- Given the scores, we can pass them through a softmax to get the attention matrix

$$\mathbf{A} = \mathcal{S}(\mathbf{S}) = \exp\left(\frac{\mathbf{S}}{\text{diag}(\mathbf{S}\mathbf{1}_N)}\right)$$

- (The expression $\mathbf{S}\mathbf{1}_N$ is a vector that sums along the columns of $\mathbf{S}$, and the division is performed elementwise.)
- Once we have computed the attention weights, we can perform the dictionary lookup for a batch of queries as follows:

$$\text{attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \mathbf{A}\mathbf{V} = \exp\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\text{diag}((\mathbf{Q}\mathbf{K}^\top)\mathbf{1}_n)}\right)\mathbf{V}$$

Soft vs hard attention

- If we force the attention heatmap to be sparse, so that each output can only attend to one input location instead of a weighted combination of all of them, the method is called hard attention.



(a)



(b)

Image captioning using attention. (a) Soft attention. Generates “a woman is throwing a frisbee in a park”. (b) Hard attention. Generates “a man and a woman playing frisbee in a field”.

Transformers

- The transformer model is a seq2seq model which uses attention in the encoder as well as the decoder, thus eliminating the need for RNNs, as we explain below.
- Transformers have been used for many sequence generation tasks, such as machine translation, physical systems, music generation, protein sequence generation, abstractive text summarization, image generation (treating the image as a rasterized 1d sequence), etc.
- The transformer is a rather complex model that uses several new kinds of building blocks or layers.
- Common models: [GPT](#) (Generative Pre-trained Transformer), [BERT](#) (Bidirectional Encoder Representations from Transformers)

Self-attention

- Earlier, we showed how the decoder of an RNN could use attention to the input sequence, in addition to using an RNN encoder, in order to capture contextual embeddings of each input.
-
- However, rather than the decoder attending the encoder, we can modify the model so the encoder attends to itself.
- Specifically, layer l of the model can attend to layer $l - 1$ of the same model. This is called **self attention**.
- This allows us to generate all the outputs in parallel, thus eliminating the need for RNN.

Transformers

- More precisely, in layer ℓ , we compute the hidden units at location t using

$$\mathbf{h}_t^\ell = \varphi(\mathbf{W}\mathbf{r}_t^\ell)$$

where $\mathbf{r}_t^\ell = \sum_s A_{ts} \mathbf{h}_s^{\ell-1}$, and

$$A_{ts} = \mathcal{S}\left(\text{score}(\mathbf{h}_t^{\ell-1}, \mathbf{h}_s^{\ell-1})\right)$$

- By stacking multiple self-attention layers, we can learn a nonlinear representation of a sequence, where the hidden representation of each word depends on the hidden representation of every other word.

Transformers

- As an example of how this can be useful:
- Consider translating the English sentence:
“*The animal didn’t cross the street because it was too tired*” into French.
- Clearly the word “it” refers to the animal.
- Now consider the slightly different English sentence
“*The animal didn’t cross the street because it was too wide*”.
- Clearly the word “it” now refers to the street.
- To generate a pronoun of the correct gender in French, we need to solve the coreference resolution problem on the input, to figure out what “it” refers to.

Transformers

- How self attention applied to the English sentence is able to resolve this ambiguity.
- In particular, in the first sentence, the representation for “it” depends on the earlier representations of “animal”, whereas in the latter, it depends on the earlier representations of “street”.

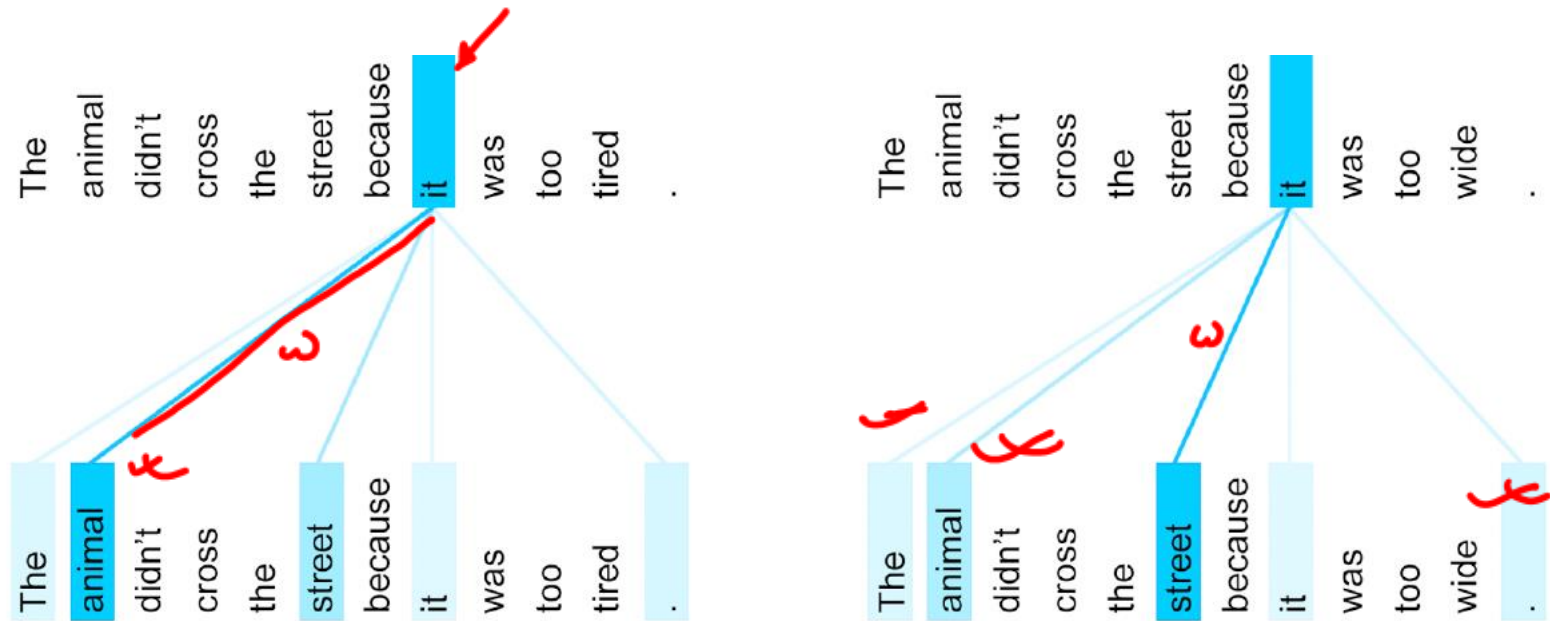


Illustration of how encoder self-attention for the word “it” differs depending on the input context.

Transformers

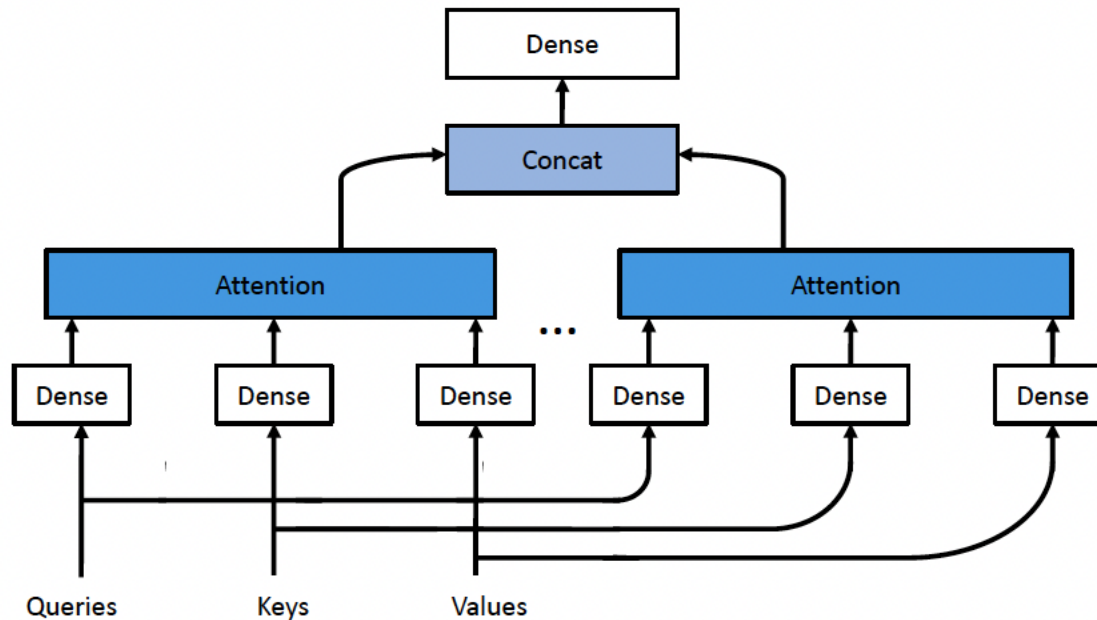
- To use self attention in a generative model, we need to use a modified version of attention, known as **masked attention**, to ensure we only look at output values that have already been generated
- This can be done by restricting the sum to only include earlier hidden units from the previous layer, i.e., $\mathbf{r}_t^\ell = \sum_{s < t} A_{ts} \mathbf{h}_s^{\ell-1}$.
- We can implement masked attention by setting the corresponding inputs to the softmax to a large negative number.

Multi-headed attention

- We can increase the expressive power of the attention layer by learning multiple versions of the dictionary, where we modify the query, keys and values using learned projection matrices.
- More precisely, let d_q, d_k and d_v be the hidden sizes to which we project the queries, keys and values, and let $\mathbf{W}_q^{(j)} \in \mathbb{R}^{d_q \times D_q}$, $\mathbf{W}_k^{(j)} \in \mathbb{R}^{d_k \times D_k}$ and $\mathbf{W}_v^{(j)} \in \mathbb{R}^{d_v \times D_v}$ be the projection matrices, for $j = 1:J$.
- Then we can compute J outputs as in parallel as follows:
$$\mathbf{r}^{(j)} = \text{attention}(\mathbf{W}_q^{(j)} \mathbf{q}, \mathbf{W}_k^{(j)} \mathbf{k}, \mathbf{W}_v^{(j)} \mathbf{v})$$
- These outputs are then combined and passed through a final projection layer. This is known as **multi-head attention**.

Multi-headed attention

- Intuitively this corresponds to comparing the queries and keys in different subspaces, and then returning modified versions of the values in response.



Multi-head attention.

Multi-headed attention

- The benefit of this approach

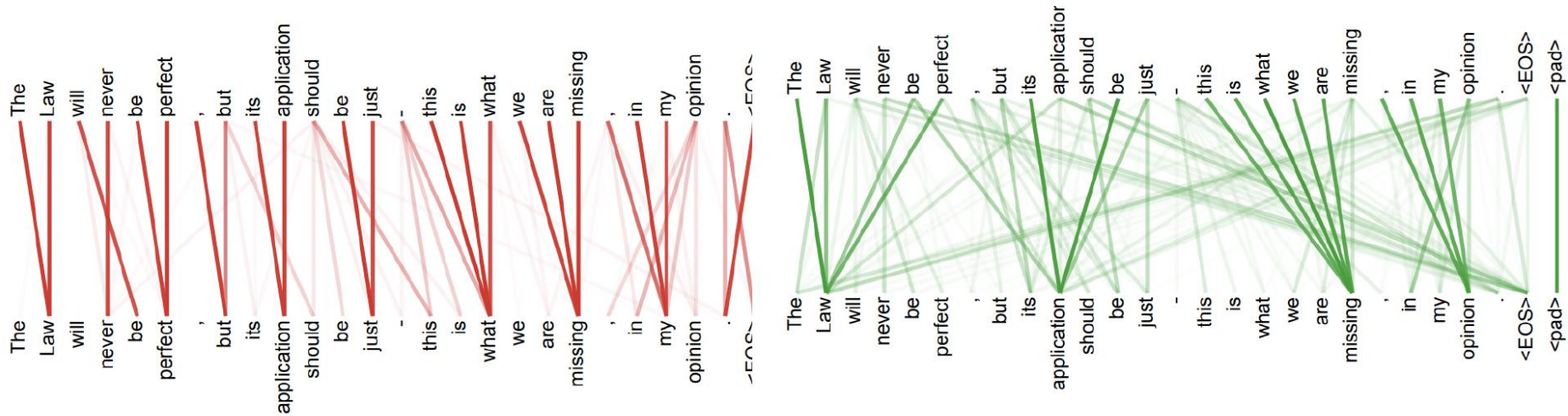


Illustration of the attention heatmaps generated by two different heads in a machine translation task.

- On the left we see one head in which each word tends to attend to its own representation in the earlier layer, since that maximizes the pairwise score.
- However, on the right we see a different heads learns to attend to contextual words.

Positional encoding

- The performance of “vanilla” self-attention can be low, since attention is permutation invariant, and hence ignores the input word ordering.
- To overcome this, we can concatenate the word embeddings with a positional embedding, so that the model knows what order the words occur in.
- In more detail, let us assume that the input to attention is a matrix $\mathbf{X} \in \mathbb{R}^{T \times D}$, where T is the length of the sequence.
- To ensure the attention model knows where each word occurs in its input, we can augment it with a positional encoding vector.

Positional encoding

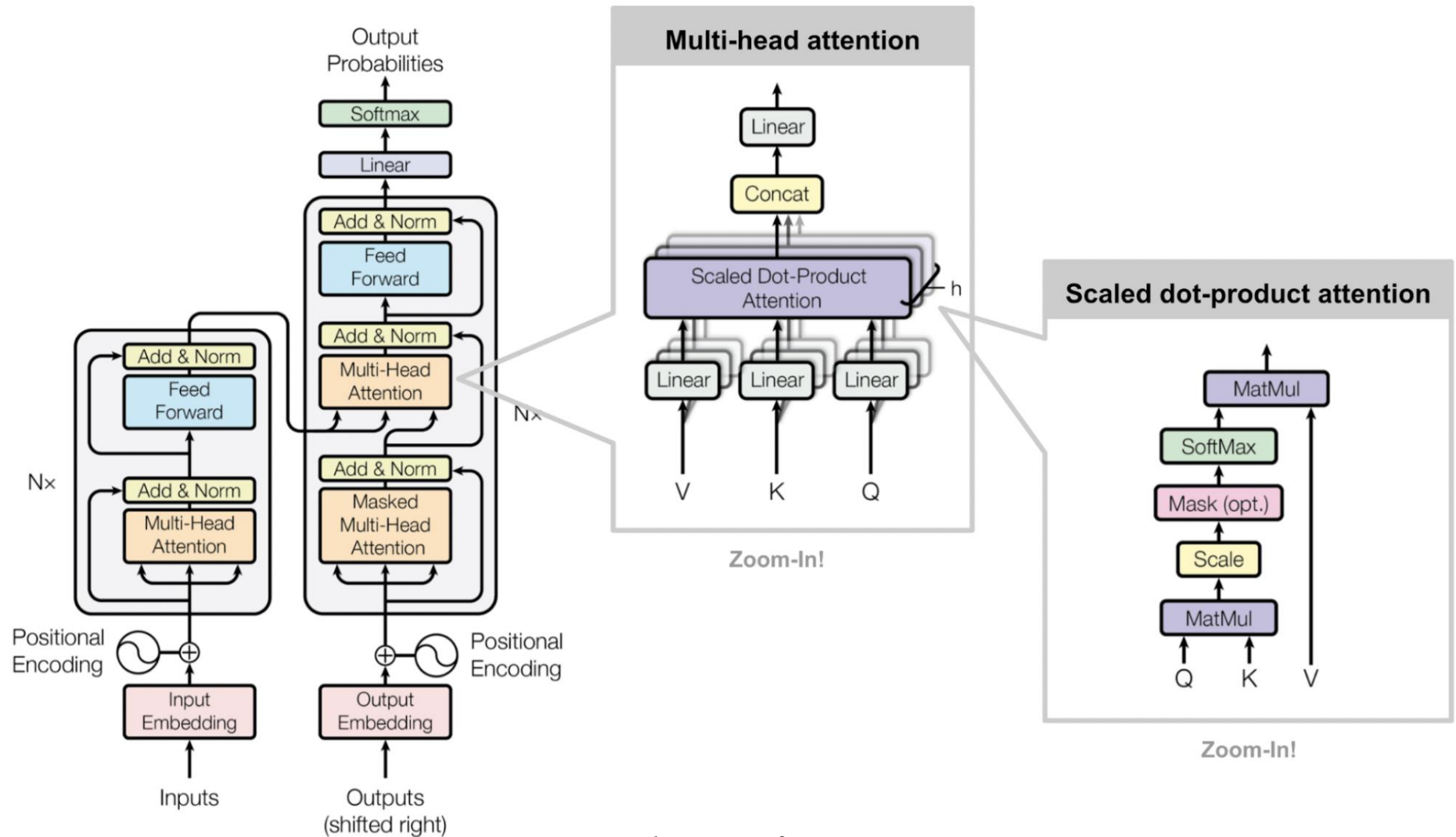
- In particular, we map the integer location indices $t \in \{1, \dots, T\}$ to a matrix of $\mathbf{P} \in \mathbb{R}^{T \times D}$.
- We could use a binary encoding of t , which would take $D = \log_2 T$ bits. However, this does not leverage the fact that the vector representation $\mathbf{p}_t$ for location t is a dense floating point representation.
- The propose the following alternative representation:
$$\mathbf{p}_t = [\sin(\omega_1 t), \cos(\omega_1 t), \sin(\omega_2 t), \cos(\omega_2 t), \dots, \sin(\omega_{d/2} t), \cos(\omega_{d/2} t)]$$

where $\omega_k = 1/C^{2k/d}$ is the frequency for the k 'th entry, d is the number of basis functions, and $C = 10,000$ is some constant.

Putting it altogether

- A transformer is a seq2seq model that uses self-attention for the encoder and decoder rather than an RNN.
- The encoder uses a series of residual blocks, each of which uses multi-headed attention and layer normalization.
- Each block $\mathbf{z} = f(\mathbf{x})$ can be defined as follows:
$$\mathbf{z} = \text{LayerNorm}(\varphi(\mathbf{W}\mathbf{x}') + \mathbf{x}')$$
$$\mathbf{x}' = \text{LayerNorm}(\text{MultiHeadAttention}(\mathbf{x}) + \mathbf{x})$$
- This is applied to the original input sequence after embedding and concatenating with positional encodings.
- The decoder has a somewhat more complex structure. It is given access to the encoder via another multi-head attention block.
- But it is also given access to previously generated outputs: these are shifted, and then combined with a positional embedding, and then fed into a masked (causal) multi-head attention model.
- Finally the output distribution over tokens at each location is computed in parallel.

Transformers



The transformer.